



第一章:引言



数据库管理系统(DBMS)

DBMS包含有关特定企业的信息z相关数据的集合

z访问数据的一组程序

z使用“数据库应用程序”既方便又高效的环境:

z银行业务:交易

z航空公司:预订、时刻表

z大学:注册, 成绩

z销售:客户、产品、采购

z线上零售商:订单跟踪、定制推荐z制造:生产、库存、订单、供应链

z人力资源:员工记录、工资、税收减免等数据库触及我们生活的方方面面



大学数据库示例

“应用程序示例”

增加新的学生、教师和课程

为学生注册课程，并生成班级花名册

给学生分配分数，计算平均绩点(GPA)并生成成绩单

在早期，数据库应用程序直接构建在文件系统之上



数据库系统的目的



使用文件系统存储数据的缺点:

- z 数据冗余和不一致

- 多种文件格式, 不同文件中的信息重复 z 数据访问困难

- 需要编写新的程序来执行每一个新的任务 z 数据隔离——多个文件和格式

- z 完整性问题

- 完整性约束(例如, 账户余额 > 0)变成

- “隐藏”在程序代码中, 而不是明确声明很难添加新的约束或更改现有的约束



数据库系统的目的(续)

☒ 使用文件系统的缺点(续)

z 更新的原子性

失败可能会使数据库在执行部分更新时处于不一致状态

示例:从一个账户到另一个账户的资金转移要么完成, 要么根本不发生

z 多用户并发访问

性能需要的并发访问

不受控制的并发访问可能导致不一致

-示例:两个人同时读取一个余额(假设100)并通过取款(假设每个人50)来更新它

z 安全问题

很难让用户访问部分(但不是全部)数据

☒ 数据库系统为上述所有问题提供了解决方案



抽象层次

物理层:描述记录(如客户)的存储方式。**逻辑层:**描述存储在数据库中的数据,以及数据之间的关系。

类型教员=记录

ID: string;

名称:string;

Dept_name: string;

工资:integer;

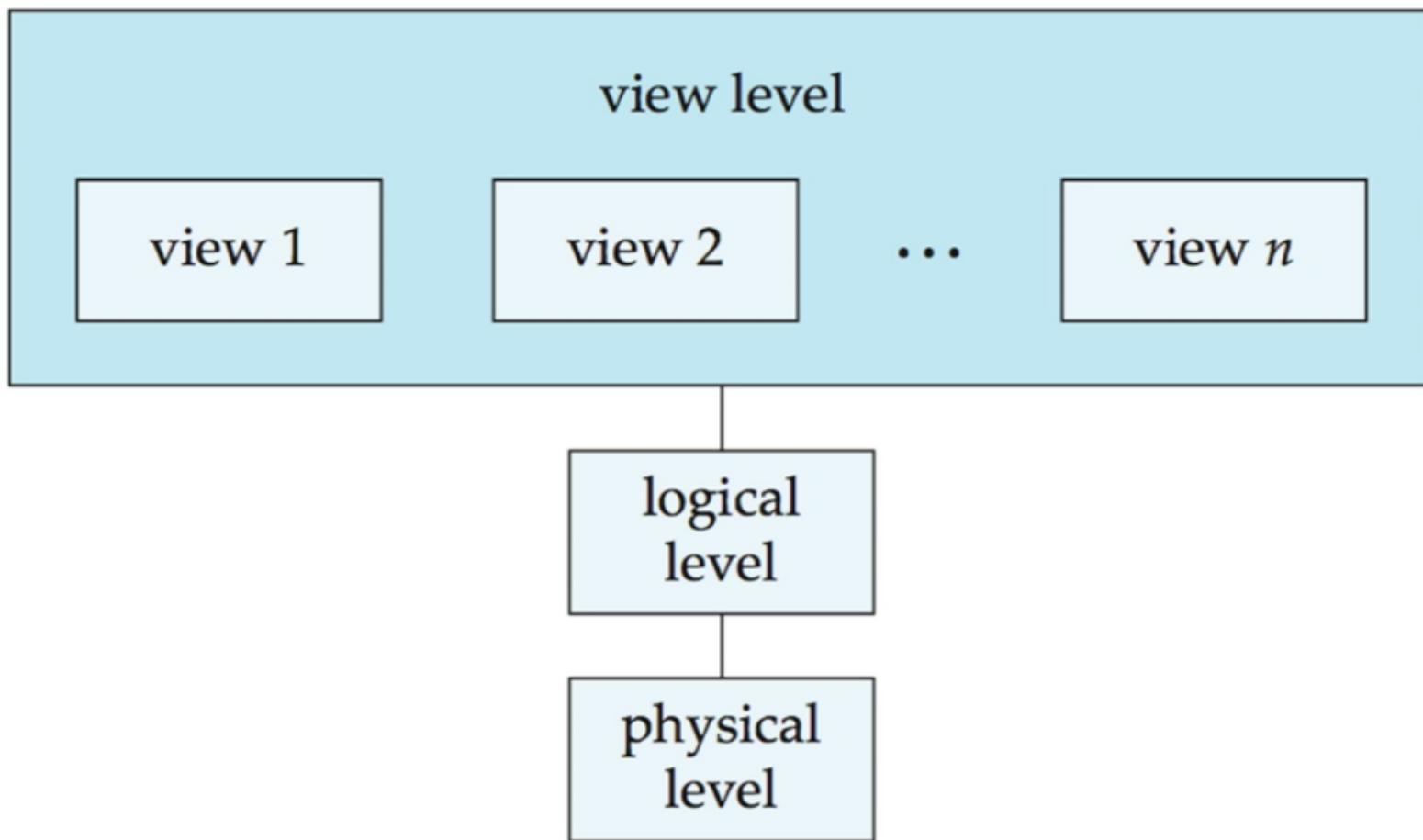
结束;

查看级别:应用程序隐藏数据类型的详细信息。出于安全考虑,视图还可以隐藏信息(如员工的工资)。



VData视图

数据库系统的体系结构





I 实例和模式

☒ 类似于编程语言中的类型和变量

☒ **模式**——数据库的逻辑结构

z 示例:数据库由一组客户和帐户的信息以及它们之间的关系组成

z 类似于程序中变量的类型信息

z **物理模式**:物理层的数据库设计

z **逻辑模式**:逻辑层面的数据库设计

☒ **实例**——数据库在特定时间点的实际内容 z 类似于变量的值

☒ **物理数据独立性**——在不改变逻辑模式的情况下修改物理模式的能力

z 应用程序依赖于逻辑模式

一般来说, 各个层次和组件之间的接口应该

明确定义, 使某些部分的变化不会严重影响其他部分。



数据模型

用于描述的工具集合

- z 数据
- z 数据关系
- z 数据语义
- z 数据约束

“关系模型”

“实体-关系”数据模型(主要用于数据库设计) “基于对象的数据模型(面向对象和对象-关系)” “半结构化数据模型(XML)”

其他较旧的模型:

- z 网络模型
- z 层次模型



Relational Model

“关系模型(第二章)”
关系模型中的表格数据示例

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table



A Sample Relational Database

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table



数据操作语言(DML)

- ☒ 用于访问和操作由适当数据模型组织的数据的语言
 - z DML也称为查询语言
- ☒ 两类语言
 - ☒ z **Procedural** -user指定需要什么数据以及如何获取这些数据
 - z **声明性(非过程性)** -user指定需要什么数据, 而不指定如何获取这些数据

SQL是使用最广泛的查询语言



数据定义语言(DDL)

☒ 用于定义数据库模式的规范符号

示例:**create table** *instructor* ()

ID **char**(5), *name* **varchar**
(20), *dept_name* **varchar**
(20), *salary* **numeric**(8,2))

“DDL编译器生成一组存储在数据字典中的表” 数据字典包含元数据
(即关于数据的数据)

z数据库模式

z完整性约束

主键(ID唯一标识教员)

引用完整性(SQL中的引用约束)-例如:任何教员元组中的*Dept_name*
值必须出现在*department relation*中

z授权



SQL

☒ **SQL**:广泛使用的非过程语言

z示例:查找ID为22222 **select** name的指导员的姓名

从教练

教练的**地方**。Id = ' 22222 '

z**选择**教官。ID, *department.dept* **from** *instructor*,

department. name

其中*instructor.dept name* = *department.dept name*和

department.预算 > 95000

应用程序通常通过z语言扩展之一访问数据库，以允许嵌入SQL

z应用程序接口(如ODBC/JDBC)，允许将SQL查询发送到数据库

*第3章，第4章和第5章



数据库设计

设计数据库总体结构的过程:

“逻辑设计——决定数据库模式”。数据库设计要求我们找到一个“好的”关系模式集合。

业务决策——我们应该在数据库中记录哪些属性?

我们应该有什么样的关系模式，属性应该如何在不同的关系模式中分布?

“物理设计”——决定数据库的物理布局



Database Design?

❌ I这个设计有什么问题吗?

<i>ID</i>	<i>name</i>	<i>salary</i>	<i>dept_name</i>	<i>building</i>	<i>budget</i>
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000



设计方法

- ☒ 归一化理论(第八章)
 - z 形式化哪些设计是坏的, 并对它们进行测试

- ☒ 实体关系模型(第7章)
 - z 将企业建模为实体和关系的集合

实体:企业中有别于其他对象的“事物”或“对象”

-用一组属性来描述

关系:几个实体之间的关联

z 用实体关系图表示的:

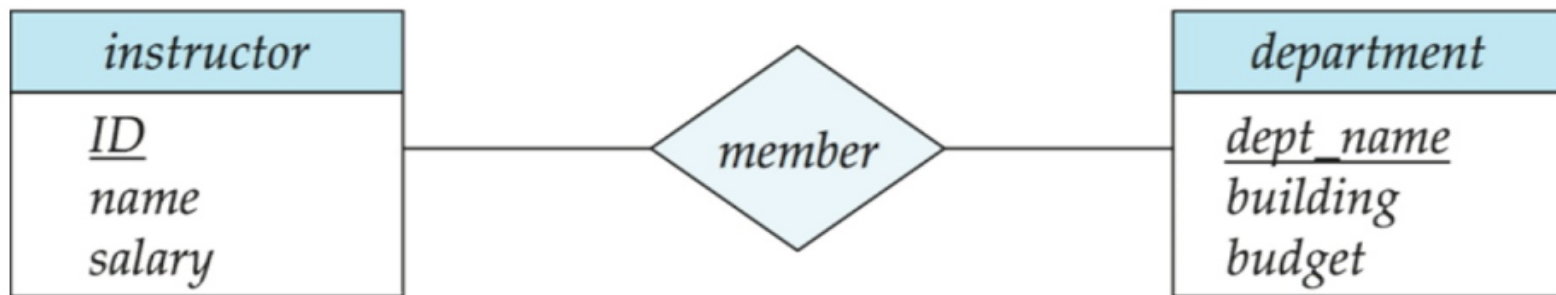


实体-关系模型

- ❑ 将企业建模为实体和关系的集合
z 实体:企业中有别于其他对象的“事物”或“对象”

由一组属性描述

z 关系:用实体关系图表示的几个实体之间的关联。



老师和学生的dept_name怎么了?



对象-关系数据模型

- ❑ 关系模型:扁平的、“原子”的值
- ❑ 对象关系数据模型
 - z 通过包括面向对象和构造来处理添加的数据类型来扩展关系数据模型。
 - z 允许元组的属性具有复杂类型，包括嵌套关系等非原子值。
 - z 保留关系基础，特别是对数据的声明式访问，同时扩展建模能力。
 - z 提供与现有关系语言的向上兼容性。



XML:可扩展标记语言

*由WWW联盟(W3C)定义

最初打算作为文档标记语言而不是数据库语言

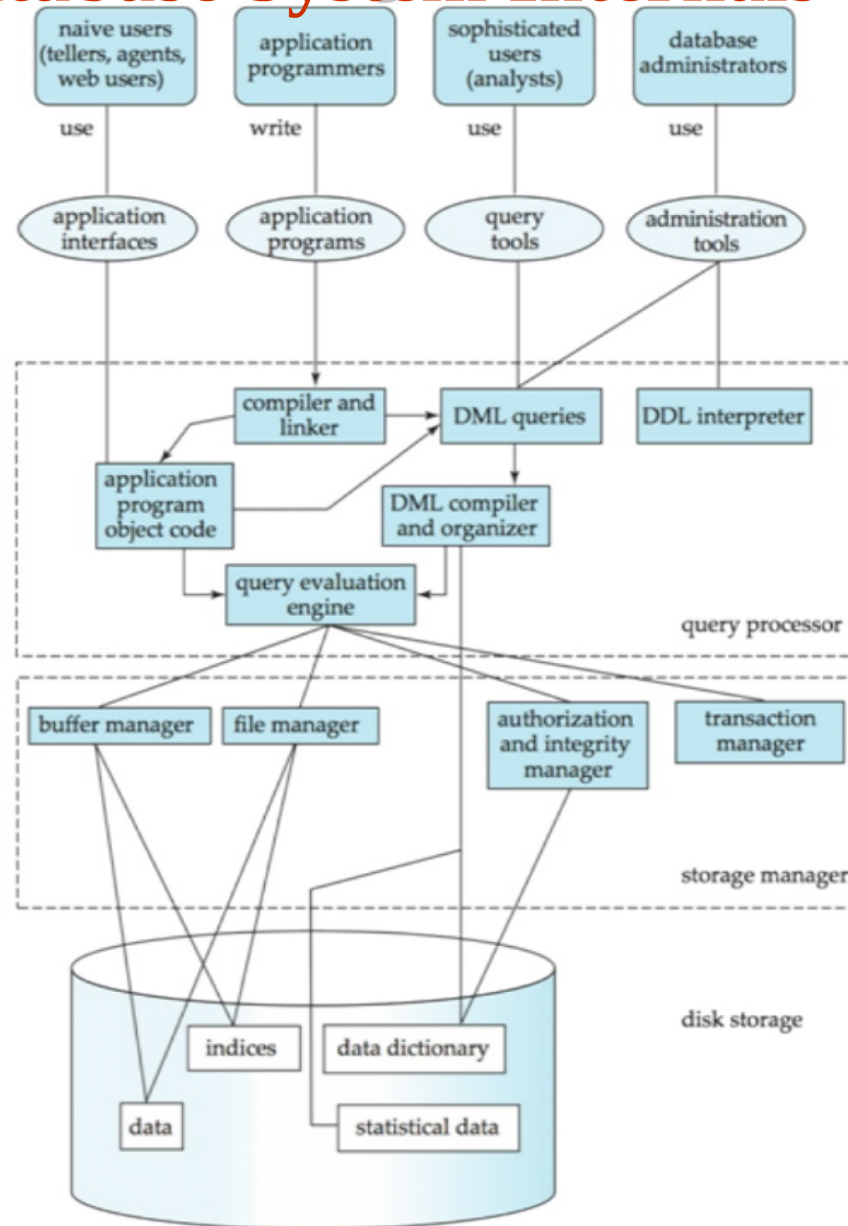
“指定新标签和创建嵌套标签结构的能力使XML成为交换数据的好方法，而不仅仅是交换文档

“XML已经成为所有新一代数据交换格式的基础。

各种各样的工具可用于解析、浏览和查询XML文档/数据



Database System Internals





存储管理

“**存储管理器**”是一个程序模块，它提供存储在数据库中的低级数据与提交给系统的应用程序和查询之间的接口。

“存储管理器”负责以下任务:

- 与文件管理器的交互

- 有效地存储、检索和更新数据

☒问题:

- 存储访问

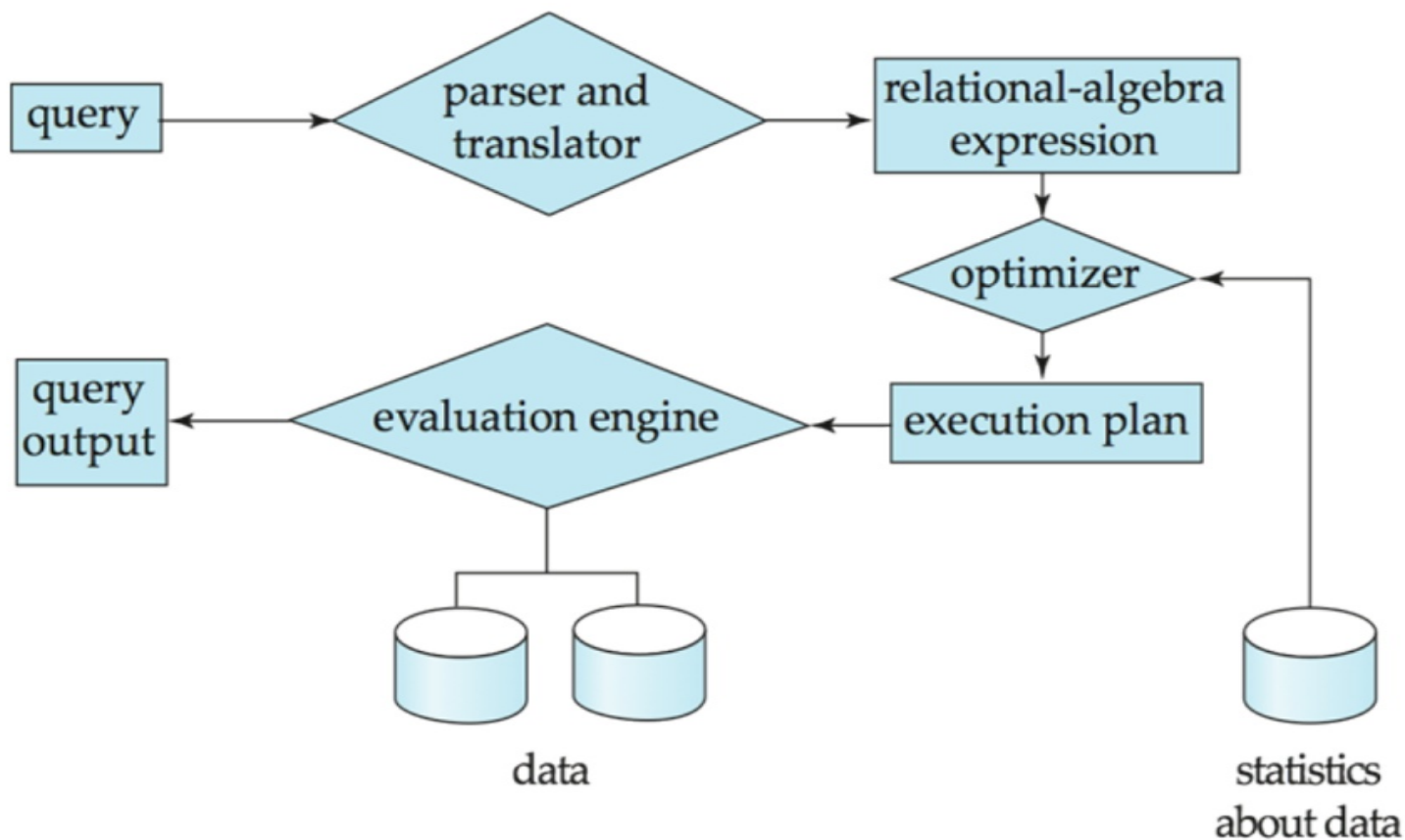
- 文件组织

- 索引和哈希



Query Processing

1. 解析与翻译
2. 优化
3. 评价





查询处理(续)

对给定查询求值的替代方法

- z 等价表达式

每个操作使用不同的算法

“评估查询的好方法和坏方法之间的成本差异可能是巨大的

需要估算操作的成本

- z 主要依赖于数据库必须维护的关系的统计信息

- z 需要估计中间结果的统计信息，以计算复杂表达式的代价



事务管理

“如果系统出现故障怎么办？

如果多个用户同时更新相同的数据怎么办？“**事务**是在数据库应用程序中执行单个逻辑功能的操作的集合

事务管理组件确保数据库在系统发生故障(例如：

电源故障和操作系统崩溃)和事务失败。**并发控制管理器**控制程序之间的交互

并发事务，保证数据库的一致性。



数据库体系结构

数据库系统的体系结构很大程度上受到运行数据库的底层计算机系统的影响:

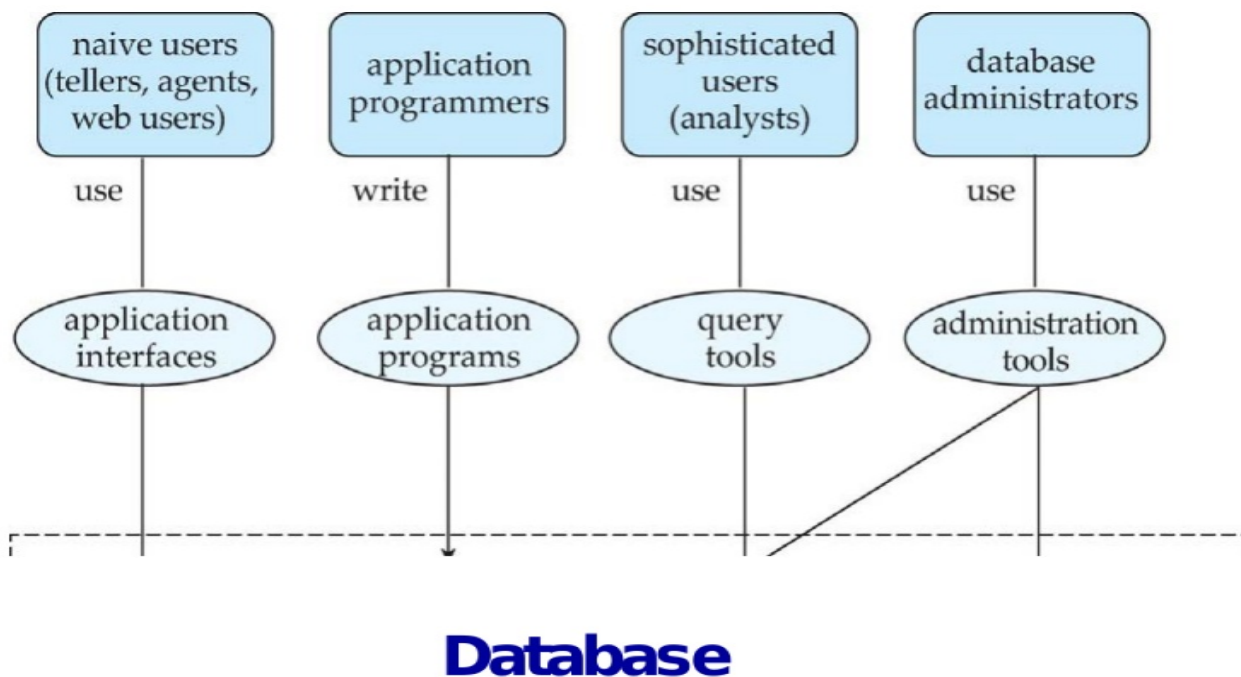
☒集中

☒客户机-服务器

并行(多处理器) “分布式”



Database Users and Administrators





数据库系统的历史

“50年代和60年代初:

- z 使用磁带进行存储的数据处理磁带只提供顺序访问
- z 用于输入的穿孔卡片

“20世纪60年代末和70年代:

- 硬盘 允许直接访问数据
- z 广泛使用的网络和分层数据模型
- z Ted Codd定义了关系数据模型将为此工作获得ACM图灵奖IBM研究院开始System R原型
- UC Berkeley开始Ingres原型
- z 高性能(对于那个时代)的事务处理



历史(续)。

☒ 1980年代:

- z 研究关系原型演变成商业系统SQL成为工业标准
- z 并行和分布式数据库系统
- z 面向对象的数据库系统

☒ 1990年代:

- z 大型决策支持和数据挖掘应用
- z 大型多tb数据仓库
- z 网络商务的出现

“21世纪初:

- z XML和XQuery标准
- 自动化数据库管理

“2000年代后期:

- z 巨型数据存储系统

谷歌BigTable, Yahoo PNuts, Amazon, ..



第一章结束



Figure 1.02

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

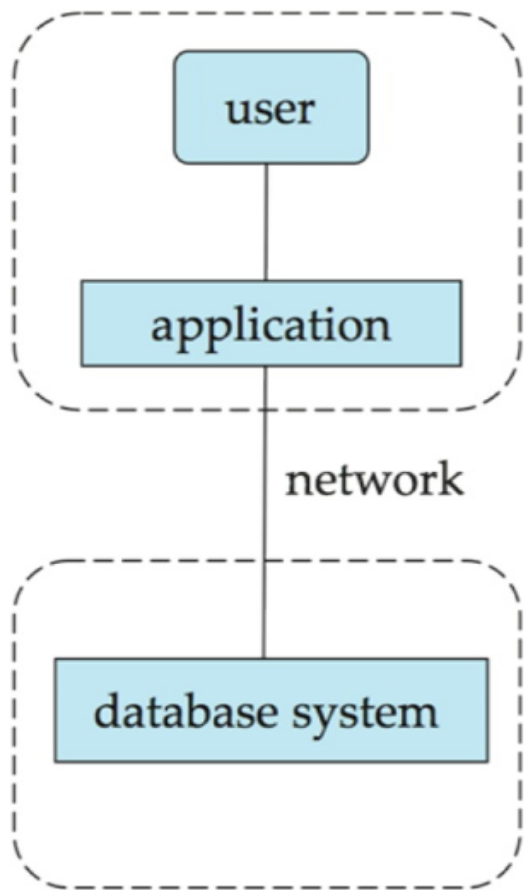


Figure 1.04

<i>ID</i>	<i>name</i>	<i>salary</i>	<i>dept_name</i>	<i>building</i>	<i>budget</i>
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

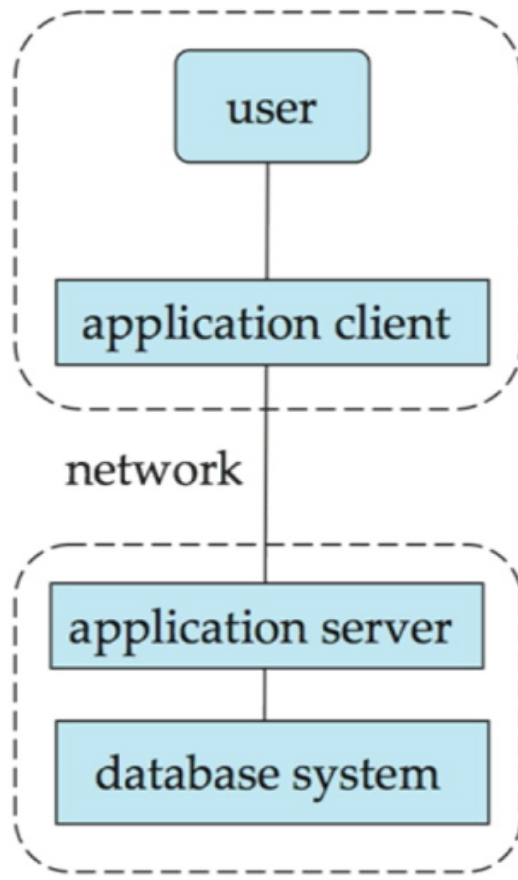


图1.06



(a) Two-tier architecture

client



(b) Three-tier architecture

server